



VIT[®]
B H O P A L
www.vitbhopal.ac.in

Fall Semester: 2025-26

Lab Experiments File

CSD3009 - Data Structures and Analysis of Algorithms

Slot – B11+B12+B13

Faculty Name - Dr. Daood Saleem

By: GROUP-1

Table of Content

S. No.	Experiment Name	Page No.	Date	Signature
1.	Write a program that uses functions to perform the following operations on singly linked list i) Creation ii) Insertion iii) Deletion iv) Traversal.	1-22	1-09-2025	

Experiment

Aim: Write a program that uses functions to perform the following operations on singly linked

list i) Creation ii) Insertion iii) Deletion iv) Traversal.

Problem Description in detail:

The goal is to design and implement a program in C that performs fundamental operations on a singly linked list. A linked list is a dynamic data structure composed of nodes, where each node contains data and a pointer to the next node in the sequence. The program must be modular, utilizing separate functions to handle each of the four core operations:

1. **Creation:** Build a new singly linked list from scratch by allowing the user to specify the number of nodes and their corresponding data.
2. **Insertion:** Add a new node to an existing linked list. This should support insertion at three different locations: at the beginning, at the end, and at a user-specified position.
3. **Deletion:** Remove a node from an existing linked list. This should support deletion from the beginning, from the end, and by searching for a specific data value.
4. **Traversal:** Traverse the linked list from the beginning (head) to the end and display the data contained in each node in sequential order.

The program should provide a user-friendly, menu-driven interface to allow for easy selection and execution of these operations.

Theory :

What is a Singly Linked List?

A singly linked list is a linear data structure where elements are not stored in contiguous memory locations. Instead, each element, called a node, consists of two parts:

- **Data:** The actual value or information stored in the node.
- **Next Pointer:** A pointer that stores the memory address of the next node in the sequence.

The list is accessed through a special pointer, commonly named head, which points to the very first node. The last node in the list has its next pointer set to NULL, signifying the end of the list.

Algorithm:

i) Creation

The creation process involves building the list from an empty state (head = NULL). The user specifies the number of nodes, and data for each node is read sequentially.

1. Initialize head pointer to NULL.
2. Ask the user for the number of nodes n.
3. Loop n times: a. Allocate memory for a newNode. b. Read the data for the newNode. c. Set newNode->next to NULL. d. If the list is empty (head == NULL): Set head to point to newNode. e. Else: Traverse the list from the head to find the last node. Set the next pointer of the last node to newNode.
4. The list is now created.

ii) Insertion

a) At the Beginning:

1. Allocate memory for a newNode.
2. Assign data to newNode->data.
3. Set newNode->next to the current head.
4. Update head to point to newNode.

b) At the End:

1. Allocate memory for a newNode.
2. Assign data to newNode->data and set newNode->next to NULL.
3. If the list is empty (head == NULL): Set head to point to newNode.
4. Else: Traverse to the last node (where temp->next == NULL).
5. Set lastNode->next to newNode.

c) At a Specific Position:

1. Check if the position is valid (e.g., pos >= 1).
2. If pos == 1, perform an "Insert at Beginning" operation.
3. Otherwise, traverse the list to the node at pos - 1. Let's call it prevNode.
4. If prevNode is NULL (position is out of bounds), show an error.
5. Allocate memory for a newNode and assign data.
6. Set newNode->next to prevNode->next.
7. Set prevNode->next to newNode.

iii) Deletion

a) From the Beginning:

1. Check if the list is empty (head == NULL). If so, there's nothing to delete.
2. Create a temporary pointer temp and point it to head.
3. Update head to point to the next node (head = head->next).

4. Free the memory pointed to by temp.

b) From the End:

1. Check if the list is empty.
2. If the list has only one node (head->next == NULL), delete the head.
3. Traverse the list with two pointers, currentNode and prevNode, until currentNode reaches the last node.
4. Set prevNode->next to NULL.
5. Free the memory of currentNode.

c) By Value:

1. Check if the list is empty.
2. If the value is in the head node, perform a "Delete from Beginning" operation.
3. Traverse the list with two pointers, currentNode and prevNode, searching for the node with the matching value.
4. If the value is found, set prevNode->next to currentNode->next.
5. Free the memory of currentNode.
6. If the end of the list is reached and the value is not found, notify the user.

iv) Traversal

1. Check if the list is empty (head == NULL).
2. Create a temporary pointer temp and initialize it with head.
3. Loop as long as temp is not NULL: a. Print the data of the node pointed to by temp. b. Move temp to the next node (temp = temp->next).

Pseudo code:

// Node Structure

STRUCTURE Node

INTEGER data

Node POINTER next

// Global Pointer

Node POINTER head = NULL

// Main Function

INTEGER choice

LOOP forever

DISPLAY menu (Create, Insert, Delete, Traverse, Exit)

READ choice

```
SWITCH choice  
  CASE 1: createList()  
  CASE 2: insertNode()  
  CASE 3: deleteNode()  
  CASE 4: traverseList()  
  CASE 5: EXIT
```

```
// Creation  
INTEGER n, i, data  
head = NULL  
PRINT "Enter number of nodes:"  
Node POINTER temp = NULL  
FOR i from 1 to n  
  PRINT "Enter data:"  
  newNode = ALLOCATE Node  
  newNode.data = data  
  newNode.next = NULL  
  IF head == NULL THEN  
    head = newNode  
    temp = head  
  ELSE  
    temp.next = newNode  
    temp = newNode
```

```
// Insertion at Beginning  
newNode = ALLOCATE Node  
newNode.data = data  
newNode.next = head  
head = newNode
```

```
// Insertion at End  
newNode = ALLOCATE Node  
newNode.data = data  
newNode.next = NULL  
IF head == NULL THEN  
  head = newNode  
  RETURN  
temp = head  
WHILE temp.next != NULL  
  temp = temp.next
```

```
END WHILE  
temp.next = newNode
```

```
// Insertion at Specific Position  
newNode = ALLOCATE Node  
newNode.data = data  
newNode.next = NULL  
IF pos == 1 THEN  
    newNode.next = head  
    head = newNode  
    RETURN  
temp = head  
INTEGER i = 1  
WHILE i < pos-1 AND temp != NULL  
    temp = temp.next  
    i = i + 1  
IF temp == NULL THEN  
    PRINT "Position out of range"  
    RETURN  
newNode.next = temp.next  
temp.next = newNode
```

```
// Deletion from Beginning  
IF head == NULL THEN  
    PRINT "List is empty"  
    RETURN  
END IF  
Node POINTER temp = head  
head = head.next  
FREE temp
```

```
// Deletion from End  
IF head.next == NULL THEN  
    FREE head  
    head = NULL  
    RETURN  
temp = head  
WHILE temp.next.next != NULL  
    temp = temp.next  
FREE temp.next
```

temp.next = NULL

// Deletion by Value

IF head.data == val THEN

temp = head

head = head.next

FREE temp

RETURN

prev = head

temp = head.next

WHILE temp != NULL AND temp.data != val

prev = temp

temp = temp.next

IF temp == NULL THEN

PRINT "Value not found"

RETURN

prev.next = temp.next

FREE temp

// Traversal

IF head == NULL THEN

PRINT "List is empty"

RETURN

Node POINTER temp = head

WHILE temp != NULL

PRINT temp.data

temp = temp.next

C Program:

#include <stdio.h>

#include <stdlib.h>

void createList();

void insertAtBeginning();

void insertAtEnd();


```

void insertAtPosition();
void deleteFromBeginning();
void deleteFromEnd();
void deleteFromPosition();
void traverse();
// Define the structure for a node in the linked list
struct Node {
    int data;
    struct Node *next;
};

// Global pointer to the first node of the list
struct Node *head = NULL;
int main() {
    int choice;

    // The main menu loop
    while (1) {
        printf("\n\n--- SINGLY LINKED LIST MENU ---\n");
        printf("1. Create List\n");
        printf("2. Display List (Traversal)\n");
        printf("3. Insert at the beginning\n");
        printf("4. Insert at the end\n");
        printf("5. Insert at a specific position\n");
        printf("6. Delete from the beginning\n");
        printf("7. Delete from the end\n");
        printf("8. Delete from a specific position\n");
        printf("9. Exit\n");
        printf("-----\n");
        printf("Enter your choice: ");
        scanf("%d", &choice);

        switch (choice) {
            case 1:
                createList();

```

```

        break;
    case 2:
        traverse();
        break;
    case 3:
        insertAtBeginning();
        break;
    case 4:
        insertAtEnd();
        break;
    case 5:
        insertAtPosition();
        break;
    case 6:
        deleteFromBeginning();
        break;
    case 7:
        deleteFromEnd();
        break;
    case 8:
        deleteFromPosition();
        break;
    case 9:
        printf("Exiting program. Goodbye!\n");
        exit(0);
    default:
        printf("Invalid choice. Please enter a number between 1 and 9.\n");
}
}

return 0;
}

void createList() {
    if (head != NULL) {

```

```

    printf("List already exists. Creating a new one will erase the old one.\n");
    // Free the old list before creating a new one (optional, but good practice)
    struct Node *temp = head, *nextNode;
    while (temp != NULL) {
        nextNode = temp->next;
        free(temp);
        temp = nextNode;
    }
    head = NULL;
}

```

```

int n, data, i;
printf("Enter the number of nodes to create: ");
scanf("%d", &n);

```

```

if (n <= 0) {
    printf("Number of nodes should be greater than 0.\n");
    return;
}

```

```

struct Node *newNode, *temp;

```

```

// Create the head node
printf("Enter data for node 1: ");
scanf("%d", &data);
head = (struct Node *)malloc(sizeof(struct Node));
if (head == NULL) {
    printf("Unable to allocate memory.\n");
    return;
}
head->data = data;
head->next = NULL;
temp = head;

```

```

// Create remaining nodes

```

```

for (i = 2; i <= n; i++) {
    newNode = (struct Node *)malloc(sizeof(struct Node));
    if (newNode == NULL) {
        printf("Unable to allocate memory.\n");
        return;
    }
    printf("Enter data for node %d: ", i);
    scanf("%d", &data);
    newNode->data = data;
    newNode->next = NULL;
    temp->next = newNode; // Link previous node with the new node
    temp = temp->next;    // Move temp to the current node
}
printf("Linked list created successfully.\n");
}

void insertAtBeginning() {
    struct Node *newNode;
    int data;

    newNode = (struct Node *)malloc(sizeof(struct Node));
    if (newNode == NULL) {
        printf("Unable to allocate memory.\n");
        return;
    }

    printf("Enter data to insert at the beginning: ");
    scanf("%d", &data);

    newNode->data = data;
    newNode->next = head; // Point new node's next to the current head
    head = newNode;      // Make the new node the new head

    printf("Data inserted at the beginning.\n");
}

```

```

void insertAtEnd() {
    struct Node *newNode, *temp;
    int data;

    newNode = (struct Node *)malloc(sizeof(struct Node));
    if (newNode == NULL) {
        printf("Unable to allocate memory.\n");
        return;
    }

    printf("Enter data to insert at the end: ");
    scanf("%d", &data);

    newNode->data = data;
    newNode->next = NULL; // The new last node's next should be NULL

    if (head == NULL) {
        // If the list is empty, the new node becomes the head
        head = newNode;
    } else {
        // Traverse to the end of the list
        temp = head;
        while (temp->next != NULL) {
            temp = temp->next;
        }
        temp->next = newNode; // Link the last node to the new node
    }

    printf("Data inserted at the end.\n");
}

void insertAtPosition() {
    struct Node *newNode, *temp;
    int pos, data, i;

```

```

printf("Enter the position to insert at: ");
scanf("%d", &pos);

if (pos <= 0) {
    printf("Invalid position.\n");
    return;
}

if (pos == 1) {
    // Position 1 is the beginning
    insertAtBeginning();
} else {
    newNode = (struct Node *)malloc(sizeof(struct Node));
    if (newNode == NULL) {
        printf("Unable to allocate memory.\n");
        return;
    }

    printf("Enter data for the new node: ");
    scanf("%d", &data);
    newNode->data = data;

    temp = head;
    // Traverse to the node just before the insertion position
    for (i = 1; i < pos - 1 && temp != NULL; i++) {
        temp = temp->next;
    }

    if (temp == NULL) {
        printf("Position out of range. Cannot insert.\n");
        free(newNode); // Free the allocated memory if insertion fails
    } else {
        newNode->next = temp->next; // New node points to the next node
        temp->next = newNode;      // Previous node points to the new node
    }
}

```

```

        printf("Data inserted at position %d.\n", pos);
    }
}

```

```

void deleteFromBeginning() {
    if (head == NULL) {
        printf("List is empty. Nothing to delete.\n");
        return;
    }
}

```

```

    struct Node *temp = head;
    head = head->next; // Move head to the second node

```

```

    printf("Deleted data: %d\n", temp->data);
    free(temp); // Free the memory of the original first node
}

```

```

void deleteFromEnd() {
    if (head == NULL) {
        printf("List is empty. Nothing to delete.\n");
        return;
    }
}

```

```

    struct Node *temp = head;
    struct Node *prev = NULL;

```

```

    if (temp->next == NULL) {
        // Only one node in the list
        printf("Deleted data: %d\n", temp->data);
        free(temp);
        head = NULL;
    } else {
        // Traverse to the end, keeping track of the previous node
        while (temp->next != NULL) {

```

```

        prev = temp;
        temp = temp->next;
    }
    prev->next = NULL; // Unlink the last node
    printf("Deleted data: %d\n", temp->data);
    free(temp); // Free the last node
}
}

```

```

void deleteFromPosition() {
    if (head == NULL) {
        printf("List is empty. Nothing to delete.\n");
        return;
    }
}

```

```

int pos, i;
struct Node *temp = head, *prev = NULL;

```

```

printf("Enter the position to delete from: ");
scanf("%d", &pos);

```

```

if (pos <= 0) {
    printf("Invalid position.\n");
    return;
}

```

```

if (pos == 1) {
    // Position 1 is the beginning
    deleteFromBeginning();
} else {
    // Traverse to the node to be deleted
    for (i = 1; i < pos && temp != NULL; i++) {
        prev = temp;
        temp = temp->next;
    }
}

```



```

    if (temp == NULL) {
        printf("Position out of range. Cannot delete.\n");
    } else {
        prev->next = temp->next; // Bypass the node to be deleted
        printf("Deleted data: %d from position %d.\n", temp->data, pos);
        free(temp); // Free the memory
    }
}
}

void traverse() {
    if (head == NULL) {
        printf("List is empty.\n");
        return;
    }

    struct Node *temp = head;
    printf("Linked List: HEAD -> ");
    while (temp != NULL) {
        printf("%d -> ", temp->data);
        temp = temp->next;
    }
    printf("NULL\n");
}

```

Output:

1. Creating the list

```

--- SINGLY LINKED LIST MENU ---
1. Create List
2. Display List (Traversal)
3. Insert at the beginning
4. Insert at the end
5. Insert at a specific position
6. Delete from the beginning
7. Delete from the end
8. Delete from a specific position
9. Exit
-----
Enter your choice: 1
Enter the number of nodes to create: 3
Enter data for node 1: 1
Enter data for node 2: 2
Enter data for node 3: 3
Linked list created successfully.

```

2. Traversing in the list

```

--- SINGLY LINKED LIST MENU ---
1. Create List
2. Display List (Traversal)
3. Insert at the beginning
4. Insert at the end
5. Insert at a specific position
6. Delete from the beginning
7. Delete from the end
8. Delete from a specific position
9. Exit
-----
Enter your choice: 2
Linked List: HEAD -> 1 -> 2 -> 3 -> NULL

```

3. Insertion at beginning

```

--- SINGLY LINKED LIST MENU ---
1. Create List
2. Display List (Traversal)
3. Insert at the beginning
4. Insert at the end
5. Insert at a specific position
6. Delete from the beginning
7. Delete from the end
8. Delete from a specific position
9. Exit
-----
Enter your choice: 3
Enter data to insert at the beginning: 0
Data inserted at the beginning.

--- SINGLY LINKED LIST MENU ---
1. Create List
2. Display List (Traversal)
3. Insert at the beginning
4. Insert at the end
5. Insert at a specific position
6. Delete from the beginning
7. Delete from the end
8. Delete from a specific position
9. Exit
-----
Enter your choice: 2
Linked List: HEAD -> 0 -> 1 -> 2 -> 3 -> NULL

```

4. Insertion at end

```
--- SINGLY LINKED LIST MENU ---
1. Create List
2. Display List (Traversal)
3. Insert at the beginning
4. Insert at the end
5. Insert at a specific position
6. Delete from the beginning
7. Delete from the end
8. Delete from a specific position
9. Exit
```

```
-----
Enter your choice: 4
Enter data to insert at the end: 4
Data inserted at the end.
```

```
--- SINGLY LINKED LIST MENU ---
1. Create List
2. Display List (Traversal)
3. Insert at the beginning
4. Insert at the end
5. Insert at a specific position
6. Delete from the beginning
7. Delete from the end
8. Delete from a specific position
9. Exit
```

```
-----
Enter your choice: 2
Linked List: HEAD -> 0 -> 1 -> 2 -> 3 -> 4 -> NULL
```

5. Insertion at random position

```
--- SINGLY LINKED LIST MENU ---
1. Create List
2. Display List (Traversal)
3. Insert at the beginning
4. Insert at the end
5. Insert at a specific position
6. Delete from the beginning
7. Delete from the end
8. Delete from a specific position
9. Exit
```

```
-----
Enter your choice: 5
Enter the position to insert at: 3
Enter data for the new node: 5
Data inserted at position 3.
```

```
--- SINGLY LINKED LIST MENU ---
1. Create List
2. Display List (Traversal)
3. Insert at the beginning
4. Insert at the end
5. Insert at a specific position
6. Delete from the beginning
7. Delete from the end
8. Delete from a specific position
9. Exit
```

```
-----
Enter your choice: 2
Linked List: HEAD -> 0 -> 1 -> 5 -> 2 -> 3 -> 4 -> NULL
```

6. Deletion form beginning

```
--- SINGLY LINKED LIST MENU ---
1. Create List
2. Display List (Traversal)
3. Insert at the beginning
4. Insert at the end
5. Insert at a specific position
6. Delete from the beginning
7. Delete from the end
8. Delete from a specific position
9. Exit
-----
Enter your choice: 6
Deleted data: 0
```

7. Deletion from end

```
--- SINGLY LINKED LIST MENU ---
1. Create List
2. Display List (Traversal)
3. Insert at the beginning
4. Insert at the end
5. Insert at a specific position
6. Delete from the beginning
7. Delete from the end
8. Delete from a specific position
9. Exit
-----
Enter your choice: 7
Deleted data: 4
```

8. Deletion at random position

```
--- SINGLY LINKED LIST MENU ---
1. Create List
2. Display List (Traversal)
3. Insert at the beginning
4. Insert at the end
5. Insert at a specific position
6. Delete from the beginning
7. Delete from the end
8. Delete from a specific position
9. Exit
-----
Enter your choice: 8
Enter the position to delete from: 3
Deleted data: 2 from position 3.
```

Conclusion:

This document has outlined the complete process for designing a C program to manage a singly linked list. By breaking down the problem into four distinct operations—Creation, Insertion, Deletion, and Traversal—we have a clear and modular approach. The provided algorithms and pseudocode serve as a robust blueprint for implementation.

Successfully implementing this program demonstrates a fundamental understanding of dynamic memory allocation, pointers, and the core principles of linked list data structures. This knowledge forms a critical foundation for tackling more advanced data structures and algorithms in computer science.